

Deep Learning based Regularization as defense against Clean Label Targeted Data Poisoning

Anil Singh Parihar

Dept. of Computer Science and Engineering
 Delhi Technological University
 Delhi, India
 parihar.anil@gmail.com

Jushaan Singh Kalra

Dept. of Computer Science and Engineering
 Delhi Technological University
 Delhi, India
 jushaan18@gmail.com

Abhinav Java

Dept. of Computer Science and Engineering
 Delhi Technological University
 Delhi, India
 java.abhinav99@gmail.com

Abstract—Adversarial examples are doctored inputs to predictive models intentionally designed with a malicious purpose – to manipulate the inference time outputs of the machine learning system. One such recent threat to Deep Neural Networks is the targeted clean label data poisoning attack, wherein the adversary modifies training samples in the dataset without modifying the original labels. A model trained on these seemingly benign examples is likely to misclassify specific queries to targeted labels. While ample research is ongoing on the ways to generate and attack deep models, there is little work in mitigating such attacks. This research work employs a simpler strategy of making the model robust to poison samples during training and study the effect of regularization techniques on such attacks. Specifically, this research work proposes a strong pruning and autoencoder-based strategies to defend against data poisoning attacks and show its effectiveness on various deep models. The proposed methods reduce attacker efficacy by 23% and can serve as strong baselines for future work on defense against such adversarial attacks.

Index Terms—Adversarial Attacks, Defense, Data Poisoning, Autoencoders, Sparsification, Deep Learning

I. INTRODUCTION

Deep Neural Networks (DNNs) can capture vital statistics from training data samples and achieve impressive performance on several tasks like classification [1], [2], object detection [3], [4], sales prediction [5], text extraction [6], etc. The ability to learn from data through a composition of linear and nonlinear functions is the key reason for the prominence of DNNs in industry and academia. Currently, deep learning is being used for applications like healthcare, business analytics, search engines, facial recognition, driverless cars, among others. However, a consequence of beating state of the art on various benchmarks has inadvertently raised privacy and security concerns due to the critical nature of the applications. A common privacy concern that is the reason for the malfunction of DNNs are Adversarial attacks [7]–[13]. In general, adversarial attacks are designed with a malicious intent whereby an adversary injects doctored inputs to a neural network to make it malfunction. Recently, an attack scheme called

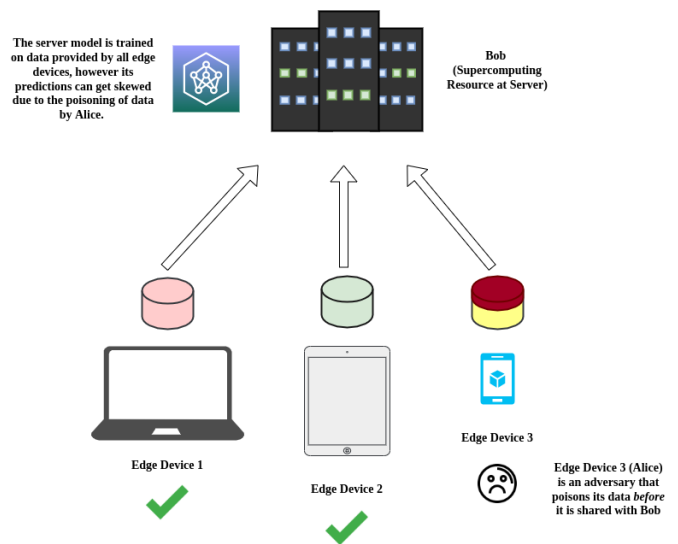


Fig. 1. A malicious adversary can add a small amount of poisoned data to the mix to manipulate the outputs of a larger server model (zoom in for best view).

Clean Label Targeted Data Poisoning [11], [12] (CLTDP) has been proposed. CLTDP shows that practically imperceptible manipulation of a very small percentage of training data can shift the decision boundary of a DNN. In contrast to inference time attacks like FSGM [8], in this attack, the adversary simply injects seemingly harmless examples (poisons) into a training dataset and achieves desired outputs for particular inputs to the model trained on this dataset. MetaPoison [10] shows that their method has an important application in watermarking images; however, they also point out the dangers of such an attack since even a small number of poisoned samples can shift the outcome by a large degree. Server models trained on client data would be particularly vulnerable to such attacks since CLTDP can work with a very small percentage of manipulated training examples. For instance (Fig. 1), consider a large

server model (Bob) and several edge devices (Alices) in a distributed learning setting [14]–[16]. Here, Bob is trained on data samples collected by Alices. If a malicious edge device injects unwanted perturbations in the shared data to manipulate the outcome of the server model, it can have several repercussions, especially in domains handling sensitive information like healthcare, politics, law, self-driving vehicles, facial recognition systems, hate speech detection, etc. With large scale models being trained on data from public sources like Twitter, Facebook, etc., it is all the more vital to create defenses against such attacks, to protect sensitive information, including the training data, model architecture, and its biases, enabling the deployment of such models for real-world use without the security concerns that currently exist.

The focus of our work is to mitigate the recently proposed attack scheme, viz Clean Label Targeted Data Poisoning, by making machine learning systems robust to potential vulnerabilities due to the CLTDP attacks. While most predominant methods have made significant strides in coming up with defenses for several types of adversarial attacks like [17]–[19], very little work has been done to mitigate CLTDP attacks. Initial methods [20], [21] mostly attempted to filter poisoned samples by identifying data samples whose features are disjoint from the rest of the data of the same class. Other techniques can broadly be divided into proxy adversarial training based [22] and differential privacy based [23]. These methods have had limited success in terms of both the attacker efficacy-utility trade-off and defense against more recent attack schemes like MetaPoison [10] that don't necessarily rely on feature manipulation for crafting poisons. We posit that a data sanitization strategy can sufficiently reduce the dimensionality of input data, effectively getting rid of the perturbations in the poisoned samples. We show that our regularization strategies are effectively able to mitigate the effects of poisoning; specifically, we are able to achieve a better tradeoff on the MetaPoison attack benchmark. The contributions of the proposed work can be summarized as follows:

- 1) We formalize the Clean Label Data Poisoning Attack scheme and discuss the existing techniques to prevent these attacks.
- 2) We design an autoencoder-based regularization technique to sanitize the poisoned data and show that it successfully prevents data poisoning on recent benchmarks.
- 3) Finally, we also discuss the effects of other regularization techniques like pruning on CLTDP and motivate the need for regularization-based solutions to prevent adversarial attacks.

The remainder of this paper is organized as - In Sec II we discuss the body of literature related to our work; this includes various CLTDP attack schemes (Sec II-A) and the mitigation strategies (Sec II-B). In Sec III-D we present our Autoencoder based regularization for preventing CLTDP attacks along with other strong baselines. Further in Sec IV, we share the experimental setup and the key findings of our work followed by conclusion and future scope in Sec V.

II. RELATED WORK

Neural Networks are vulnerable to attacks after deployment [7]–[13]. This allows adversaries to manipulate outputs based on certain triggers. One of such attacks - the targeted data poisoning is triggered by a specific pre-decided data.

A. Data Poisoning Attacks

There has been an immense amount of work around such "attacks" which can compromise the ability of a model to perform its task. Mostly, literature has focused on evasion attacks, which involve perturbing inputs at inference to get a different prediction from the model. Similar to this, backdoor attacks add a "keyword", which can trigger an unexpected output from the attacked model. However, both attacks require control over the data at inference time. Data poisoning, on the other hand, attacks data at training itself and does not require any interference at inference time. In this case, the hacker modifies or replaces some data which is to be used by the model for training. This poisoned data then leads to a compromised training pipeline. Adding this poisoned data is easy, as increasingly data is scraped from the internet by bots and used to train such models. This data could also be poisoned when it is provided by anonymous individuals, such as for Face Identification and recognition systems which are currently being used in industry [24]–[26]. Data poisoning consists of two types - one allowing the change in label as well as training input, while the other, called clean label data poisoning, allows changes in the input but with the label intact. In these attacks, the poisoned data is imperceptible from the unpoisoned one and so is difficult to remove from the training data. Moreover, these changes in the input images cause misclassification at inference time on one target image that is pre-decided. This makes defense against such systems even more difficult. There are various different Clean label data poisoning attacks that have been used against Deep learning models. MetaPoison [10] crafts a poison dataset for this purpose. They find the optimal poison images by a bilevel meta-learning approach. The novelty in this paper is that they approximate the inner optimization loop of the meta-learning pipeline by only running the SGD for K steps in the inner loop and making it tractable. The data is crafted using an adversarial approach. Intriguingly, their poisons do not collide with the target, implying that they employ some other mechanism to achieve the same objective. They do not even reside in the target class distribution, which may render neighborhood conformity tests such as Deep KNN [20] less effective as a defense. They use the CIFAR-10 [27] dataset to check their algorithm in the real world while making use of a wide variety of models such as Resnet 20 [28], VGG19 [29], Alexnet [30], LeNet [31] for poison crafting. They show high accuracy of success in both training from scratch and transfer learning. Similar to MetaPoison, PoisonGAN [32] proposes constructing poisoned data based on regeneration of victim class using GANs. They use this method to attack federated learning systems by updating global parameters using a poisoned model

from the client. They use MNIST and CIFAR-10 datasets to show the efficacy of their method against federated systems.

B. Defense against Data Poisoning Attacks

Most defenses work against a particular type of attack and are unable to defend against a wide variety of clean label attacks. Existing literature has broadly approached this problem through adversarial training (De-Pois [22]), differential privacy (DP-Aug [23]), or feature manipulation (Deep KNN [20]). These methods have had limited success in terms of both the privacy-utility trade-off and the range of attacks they can defend against and are mostly reliant on identifying the poisons. Deep KNN [20] assumes that the data poisoning is done through feature manipulation [11] and essentially removes the anomalous samples by finding the samples whose feature space is not with the rest of its class labels. CLP with LGM [21] replaces the softmax activation in the last layer with an LGM loss, building upon a very similar intuition that the features of the poison samples are actually far away than claimed (from the features of other samples of the same class). LGM loss implicitly allows the network to capture the deviation from a gaussian distribution. Both [20] and [22] fail to produce optimal results if the attack scheme is changed to something other than feature collision [11] or convex polytope [33]. DP-Aug [23] argues that mixup augmentations concentrate training data near the center of the ambient unit hypercube and saturates this region of space more densely than the original dataset, leveraging data augmentations through differential privacy they are able to secure the dataset. The key issue with this work remains that these augmentations come at the cost of poor test accuracy. De-Pois [22] uses a more expensive approach of crafting poisons using a cGAN [34] to facilitate data augmentation. However, the metric to identify poisoned samples (the difference between the performance of the mimic model and the original model) is not only computationally expensive but also uninterpretable.

III. METHOD

In this section, we first mathematically formulate CLTDP attacks, then we present the proposed Autoencoder-based regularization technique along with pruning baseline to mitigate clean label targeted data poisoning.

A. Preliminaries

Clean Label Data Poisoning was introduced by [4]. In this section we will provide the relevant background and related attack schemes. For a given training dataset at the server (Bob), let the client adversary (Alice) manipulate a subset of this dataset as such that:

$$S_{train} = S_p \cup S_c \quad (1)$$

Where S_{train} is the entire training dataset and S_p, S_c are the poisoned subset and clean subset. Note that both the subsets have correct labels in this setting. However, the subset S_c is the perturbed subset that is added maliciously and without

the practitioner's (Charlie) knowledge. The poisoned subset consists of:

$$S_c = (\tilde{X}_i, Y_i)_{i=0 \dots (n_p-1)} \quad (2)$$

Where $\tilde{X}_i$ is the perturbed sample injected into the original dataset and Y_i is the corresponding clean label. The number of poisoned samples is denoted by n_p . Let the total dataset size be n_{train} . Then generally $n_p \ll n_{train}$, making it very hard to filter the data. Furthermore, in this setting the adversary usually targets the misclassification of a particular image to the desired label. For instance, if an adversary wants to maliciously manipulate a traffic camera to not trigger when their face is inputted to this machine learning system, they can change data such that only that one input is misclassified.

In the case of the CIFAR10 dataset this problem can be simplified as follows. A target class L_k where $k \in [0, 9]$ is picked, this is the adversarial label that we want some input query q_j to be misclassified to and j is the actual class of the input query as follows:

$$k = \arg \max (f(q_j)) \quad (3)$$

Where f is the victim model and $\arg \max$ is taken over the output probabilities to output the predicted class which is $k(\text{incorrect})$ in this case.

$$i = \arg \max f(q_i) \quad (4)$$

When $i \neq j$, the victim model predicts the correct output class i .

For example: A target class can be "frog" and a query class can be "airplane". With this setup, particular airplanes are likely to be mis-classified as "frog".

B. Problem Requirements

We need to modify the victim model f such that it becomes robust to adversarial perturbations during train time. Let f be originally trained as follows:

$$\text{minimize}(L_{BCE}(f(X_i, Y_i; \theta))) \quad (5)$$

Where $i \in [0, (n_{train} - 1)]$ and the victim model is parameterized by θ and $(X, Y) \in S_{train(clean)}$. However due to CLTDP, X_i is modified as shown above which changes the decision boundary of f . We therefore reformulate the optimization problem as follows:

$$\text{minimize}(L_{BCE}(p(f(S_{train}; f); p))) \quad (6)$$

Where p is an additional learnable model that acts both as a protective model for the victim. We refer to it as a trainable data sanitizer.

C. Mitigating Data Poisoning using Pruning

Neural Networks are highly overparameterized [35] and have abundant unused parameters. Our initial hypothesis was that due to these excessive neurons, the DNN becomes more susceptible to CLTDP attacks. Pruning has shown to be effective against other types of adversarial attacks [36]. Similar to [36], wherein pruning was used to counter inference time

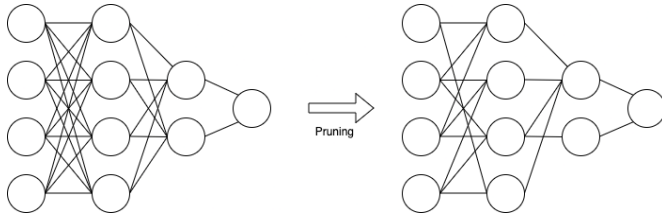


Fig. 2. A pruned DNN, is able to reduce the effectiveness of CLTDP attacks by forcing the network to share neurons to generate a representation of the input making it harder for a small group of adversarial samples to affect its decision boundary.

attack (like the adversary injecting noise during test time), we did global unstructured pruning of a network trained on the poisoned dataset to verify if there is a noticeable performance drop before and after pruning. This is our baseline model and will be used to compare against other pruning strategies. Pruning allows a deep learning practitioner to remove weights/parameters (Fig. 2) from a DNN without affecting its accuracy to a large extent. This helps the practitioner to cut deployment costs by using a smaller model. Furthermore, [37] shows that pruning is an effective regularization method as well. Optimal Brain Damage [38] argues that although the magnitude is a viable metric to judge which neurons to remove, a better method would be to use the double differential as an approximation of saliency (importance). Since then, several works have shown the benefits of both; pruning before and after training, with [39] showing that dynamic pruning followed by fine-tuning can sometimes even help achieve better accuracy. In 2019, [40] gave striking empirical results that training the best-pruned model from scratch (by resetting its weights) is actually the best method to outperform the model's overparameterized (un-pruned) counterpart. Since then, researchers have also shown interesting studies suggesting that finding the best-pruned model (or lottery ticket) is actually similar to Neural Architecture Search (NAS). In order to motivate and explore the impact of pruning on the attack efficacy, we set up a simple baseline with global unstructured pruning. This is the simplest form of pruning that globally removes some percentage of weights based on their L1 distance from zero, i.e., magnitude-based. This pruning is done after training, and the model is not fine-tuned after pruning. (as shown in Fig. 4) This essentially amounts to forcing the network to share neurons to learn a richer representation of the input. Such that, even if a certain group of adversarial examples attempts to skew a subset of weights, the other (pristine) examples preserve their correct representation.

D. Mitigating Data Poisoning using Autoencoders

Machine Learning (ML) practitioners have been using regularization methods to tackle the problem of balancing bias and variance in ML systems. Common regularization methods include Dropout [41], Weight Decay [42], Label Smoothing [43], Pruning [37], etc. In essence, all these methods are designed to prevent overfitting on the training data and improve performance on the test examples. We hypothesize that the

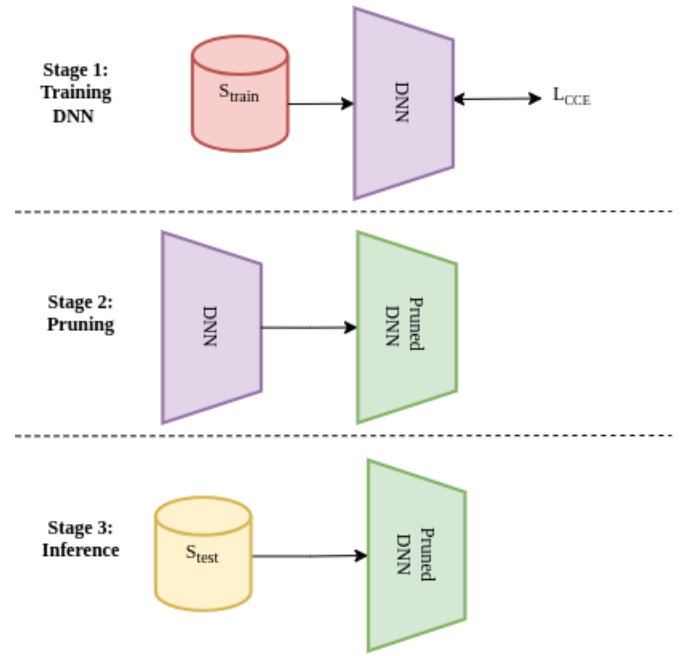


Fig. 3. System Architecture for pruning a neural network

likelihood of a DNN malfunctioning due to CLTDP attack is correlated with the dimensionality of the input training data. We sought to reduce the dimensionality of the input using a convolutional autoencoder in order to decrease the efficacy of the attacker. Specifically, the protective model p in Eq. (6) is an autoencoder [33]. Autoencoders are a class of DNNs that are used to create a more compact representation by adding a bottleneck layer that encodes the most important information needed to recreate the initial representation. (Fig. 4) It is an unsupervised technique that forms the basis of representation learning. An autoencoder can be thought of as an extension to PCA. It reduces dimensions while conserving the non-linearity of the feature space. A simple Mean Squared Error (MSE) loss can be applied between the input and the output to train an autoencoder. The compressed representation will lose the information encoded by the data poisoning attack and thereby act as an unpoisoning step before actually training a network for our original problem statement. The training is divided into two steps. We assume that the data available to the ML practitioner is already poisoned Eq. 1. We use the poisoned data to train an autoencoder p_{AE} in an unsupervised manner in order to capture the compressed representation of the data. The trained autoencoder p_{AE} is then frozen and used as p in Eq. (6) effectively acting as a protective data sanitizer over the victim model. (as shown in Fig. 5)

IV. EXPERIMENTS

In this section we highlight the experimental details like preparing the proxy poisoned data (Sec. IV-A.), evaluation protocol (Sec. IV-B.), implementation details (Sec. IV-C.) for reproducibility and finally the effectiveness of our method as results (Sec. IV-D.).

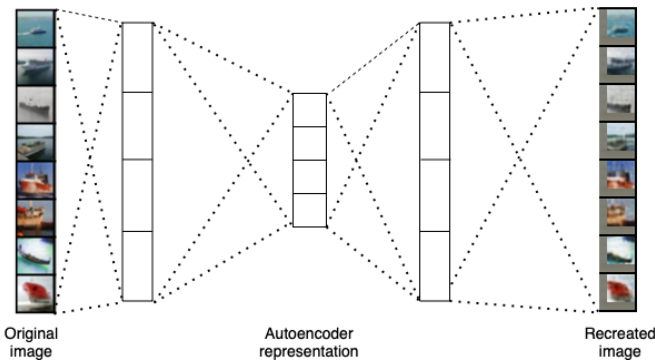


Fig. 4. Autoencoder reduces the dimensionality and is successfully able to sanitize the input data by removing the poisoning artifacts added by a malicious adversary (here Meta Poison [10]).

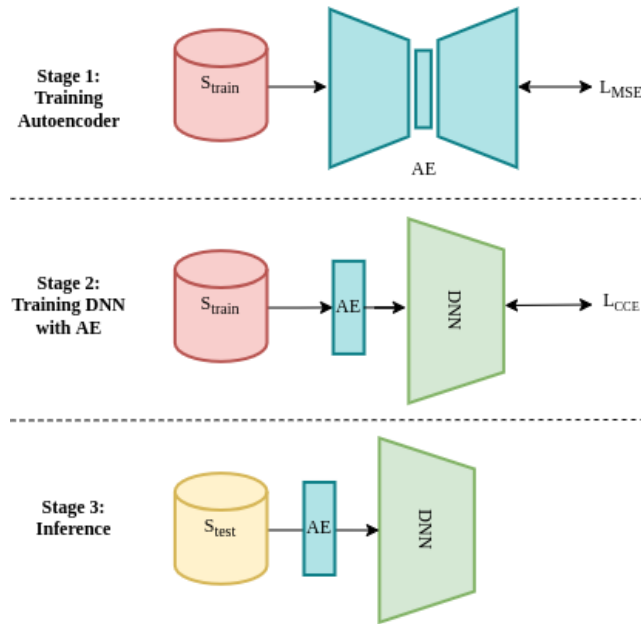


Fig. 5. System architecture for training Neural Networks with Autoencoders.

A. Dataset Preparation

In this section, we describe the preparation of data used for our CLTDP setup. We take MetaPoison [10] to first poison the CIFAR-10 [27] dataset. MetaPoison uses an approximation of a bijective function to find the optimal poison images by a meta-learning approach. However, due to computational constraints, running the bijective function is not feasible. So, the authors approximate the inner optimization loop of the meta-learning pipeline by only running the SGD for K steps in the inner loop and making it not intractable. The data is crafted using an adversarial approach. These slight perturbations in one class of the dataset make the prediction of the target image at inference time incorrect while ensuring that the changed dataset is imperceptible from the original one. Using this methodology, we construct 24 poisoned datasets from the CIFAR-10 dataset. Firstly, these datasets can be classified by choice of poison class and the targeted class. For each of these

classes, 12 models were trained.

- 1) Poison-dog-target-bird
- 2) Poison-frog-target-plane

In the first case, the model takes a subset of dataset from class dog and perturbs it, such that an image of a specific bird (say parrot) selected from the test dataset predicts the Target label defined while construction. Similarly, we use the same foundations for the construction of the 2nd dataset. For each of the above cases, we can choose a target label. For our purposes we choose 3 labels - plane, cats and frog. Next, we take 50, 500, 1000, 5000 images to be poisoned for each of the previous cases. Together these created 24 different training configurations. For each of the 24 models trained we prune 10%, 20%, 40%, 60%, 80%, 100% of the total neurons, which gives us 144 different models after pruning.

B. Evaluation Method

After training our models, we evaluate the success of our approach using two metrics – defense against CLTDP attack and the validation accuracy of the resulting model. Defense against attacks is measured by calculating the average number of times the adversarial prediction is not equal to the predicted one. The validation accuracy is determined by the average accuracy of all models on the test dataset. These results are shown in Table 1. We can observe the clear trade-off between validation accuracy and successful defense against CLTDP attacks. This is due to the nature of such defenses, they reduce information used by the model, thereby degrading its overall performance but protecting the model against unexpected attacks. Finding the fine-line between these two trade-offs is a challenging problem that could be explored in future works.

C. Implementation Details

We perform all our experiments on 1 Nvidia Titan RTX GPU using PyTorch [44]. We train the ResNet [28] models for 100 epochs and AlexNet [30] for 25 epochs. The learning rate is 0.001, while a batch size of 64 is maintained. We use the Adam solver [45] in all our experiments.

D. Results

From the results shown in Table 1, we can see that both our strategies tremendously reduce the attacker efficacy. Pruning gives 10% protection against Meta Poison [10] while maintaining a reasonable validation accuracy. Our hypothesis that strong regularization can help defend against such attacks is further validated by the fact that the best performance is on AlexNet which is inherently a smaller architecture. We note that autoencoders as data sanitizers give the best tradeoff in terms of attacker efficacy and validation accuracy of the base model. Since autoencoders offer a direct regularization effect on the data without manipulating the base model, it outperforms pruning by giving a 23% reduction in attacker accuracy.

TABLE I

EFFECTS OF PRUNING AND AUTOENCODERS ON POISONING EFFICACY. WHILE META POISON [10] WORKS 50% OF THE TIME, OUR METHOD IS ABLE TO REDUCE ITS EFFICACY BY 23%, MAKING IT MORE ROBUST.

Attack	Model	Success against poisoning attacks	Number of trained models	Val accuracy
MetaPoison	Average	49.98%	432	80.03%
	ResNet20	45.14%	144	83.46%
	ResNet32	48.55%	144	83.50%
	AlexNet	56.25%	144	73.29%
MetaPoison + Pruning	Average	59.65%	432	64.28%
	ResNet20	55.55%	144	64.51%
	ResNet32	59.51%	144	65.43%
	AlexNet	63.89%	144	62.90%
MetaPoison + Autoencoder	Average	73.61%	72	61.43%
	ResNet20	70.83%	24	62.33%
	ResNet32	75%	24	62.20%
	AlexNet	75%	24	59.86%

V. CONCLUSION

The goal of this work was to formalize Clean Label Targeted Data Poisoning and devise strategies to make the existing DNNs more robust to these attacks. Given the ever increasing use of DNNs in critical areas like healthcare, surveillance, etc and the nature of these attacks, it is imperative to protect sensitive data and make models invariant to adversaries. Since predominant works fail to defend against recent attacks (Meta Poison [10]), we design regularization-based techniques that act as generic data sanitizers and strong baselines for a diverse set of models. Our experiments show the effectiveness of the proposed method and pave the way for future research in this domain.

REFERENCES

- [1] B. Pandey and N. Garg, "Swarm optimized modular neural network based diagnostic system for breast cancer diagnosis," *Int. J. Soft Comput. Artif. Intell. Appl. (IJSCAI)*, vol. 2, no. 4, pp. 11–20, 2013.
- [2] N. A. Mohamad, N. A. Jusoh, Z. Z. Htike, S. L. Win *et al.*, "Bacteria identification from microscopic morphology: a survey," *International Journal on Soft Computing, Artificial Intelligence and Applications (IJSCAI)*, vol. 3, no. 1, pp. 2319–1015, 2014.
- [3] S. Mittal, "Unsupervised learning models of invariant features in images: recent developments in multistage architecture approach for object detection," *Int J Soft Comput Artif Intell Appl (IJSCAI)*, vol. 5, no. 2, pp. 63–70, 2016.
- [4] J. P. Verma, S. Agrawal, B. Patel, and A. Patel, "Big data analytics: Challenges and applications for text, audio, video, and social media data," *International Journal on Soft Computing, Artificial Intelligence and Applications (IJSCAI)*, vol. 5, no. 1, pp. 41–51, 2016.
- [5] A. Massaro, V. Vitti, and A. Galiano, "Model of multiple artificial neural networks oriented on sales prediction and product shelf design," *International Journal on Soft Computing, Artificial Intelligence and Applications (IJSCAI)*, vol. 7, no. 3, pp. 1–19, 2018.
- [6] A. Thilagavathy, K. Aarthi, and A. Chilambuchelvan, "Text detection and extraction from videos using ann based network," *International Journal on Soft Computing, Artificial Intelligence and Applications (IJSCAI)*, vol. 1, no. 1, 2012.
- [7] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.
- [8] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

- [9] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: a simple and accurate method to fool deep neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2574–2582.
- [10] W. R. Huang, J. Geiping, L. Fowl, G. Taylor, and T. Goldstein, "Metapoisson: Practical general-purpose clean-label data poisoning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 12080–12091, 2020.
- [11] A. Shafahi, W. R. Huang, M. Najibi, O. Suci, C. Studer, T. Dumitras, and T. Goldstein, "Poison frogs! targeted clean-label poisoning attacks on neural networks," *Advances in neural information processing systems*, vol. 31, 2018.
- [12] P. W. Koh and P. Liang, "Understanding black-box predictions via influence functions," in *International conference on machine learning*. PMLR, 2017, pp. 1885–1894.
- [13] A. Ghiasi, A. Shafahi, and T. Goldstein, "Breaking certified defenses: Semantic adversarial examples with spoofed robustness certificates," *arXiv preprint arXiv:2003.08937*, 2020.
- [14] J. Verbracken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeier, "A survey on distributed machine learning," *ACM Computing Surveys (CSUR)*, vol. 53, no. 2, pp. 1–33, 2020.
- [15] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.
- [16] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [17] K. Ren, T. Zheng, Z. Qin, and X. Liu, "Adversarial attacks and defenses in deep learning," *Engineering*, vol. 6, no. 3, pp. 346–360, 2020.
- [18] F. Boenisch, P. Sperl, and K. Böttinger, "Gradient masking and the underestimated robustness threats of differential privacy in deep learning," *arXiv preprint arXiv:2105.07985*, 2021.
- [19] B. Lindqvist and R. Izmailov, "Minimax defense against gradient-based adversarial attacks," *arXiv preprint arXiv:2002.01256*, 2020.
- [20] N. Peri, N. Gupta, W. R. Huang, L. Fowl, C. Zhu, S. Feizi, T. Goldstein, and J. P. Dickerson, "Deep k-nn defense against clean-label data poisoning attacks," in *European Conference on Computer Vision*. Springer, 2020, pp. 55–70.
- [21] M. Yaseen, M. Aadil, and M. Sargsyan, "Preventing clean label poisoning using gaussian mixture loss," *arXiv preprint arXiv:2003.00798*, 2020.
- [22] J. Chen, X. Zhang, R. Zhang, C. Wang, and L. Liu, "De-pois: An attack-agnostic defense against data poisoning attacks," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3412–3425, 2021.
- [23] E. Borgnia, J. Geiping, V. Cherepanova, L. Fowl, A. Gupta, A. Ghiasi, F. Huang, M. Goldblum, and T. Goldstein, "Dp-instahide: Provably defusing poisoning and backdoor attacks with differentially private data augmentations," *arXiv preprint arXiv:2103.02079*, 2021.
- [24] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 815–823.
- [25] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 4690–4699.
- [26] Y. Sun, C. Cheng, Y. Zhang, C. Zhang, L. Zheng, Z. Wang, and Y. Wei, "Circle loss: A unified perspective of pair similarity optimization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 6398–6407.
- [27] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [29] J. Verbracken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeier, "A survey on distributed machine learning," *ACM Computing Surveys (CSUR)*, vol. 53, no. 2, pp. 1–33, 2020.
- [30] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.
- [31] R. Zaheer and H. Shaziya, "Gpu-based empirical evaluation of activation functions in convolutional neural networks," in *2018 2nd International*

Conference on Inventive Systems and Control (ICISC). IEEE, 2018, pp. 769–773.

- [32] J. Zhang, B. Chen, X. Cheng, H. T. T. Binh, and S. Yu, “Poisoning: Generative poisoning attacks against federated learning in edge computing systems,” *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3310–3322, 2020.
- [33] C. Zhu, W. R. Huang, H. Li, G. Taylor, C. Studer, and T. Goldstein, “Transferable clean-label poisoning attacks on deep neural nets,” in *International Conference on Machine Learning*. PMLR, 2019, pp. 7614–7623.
- [34] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” *arXiv preprint arXiv:1411.1784*, 2014.
- [35] K. A. Sankararaman, S. De, Z. Xu, W. R. Huang, and T. Goldstein, “The impact of neural network overparameterization on gradient confusion and stochastic gradient descent,” in *International conference on machine learning*. PMLR, 2020, pp. 8469–8479.
- [36] X. Yuan, P. He, Q. Zhu, and X. Li, “Adversarial examples: Attacks and defenses for deep learning,” *IEEE transactions on neural networks and learning systems*, vol. 30, no. 9, pp. 2805–2824, 2019.
- [37] H. Wang, C. Qin, Y. Zhang, and Y. Fu, “Neural pruning via growing regularization,” *arXiv preprint arXiv:2012.09243*, 2020.
- [38] Y. LeCun, J. Denker, and S. Solla, “Optimal brain damage,” *Advances in neural information processing systems*, vol. 2, 1989.
- [39] S. Han, J. Pool, J. Tran, and W. Dally, “Learning both weights and connections for efficient neural network,” *Advances in neural information processing systems*, vol. 28, 2015.
- [40] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” *arXiv preprint arXiv:1803.03635*, 2018.
- [41] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [42] A. Krogh and J. Hertz, “A simple weight decay can improve generalization,” *Advances in neural information processing systems*, vol. 4, 1991.
- [43] R. Müller, S. Kornblith, and G. E. Hinton, “When does label smoothing help?” *Advances in neural information processing systems*, vol. 32, 2019.
- [44] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [45] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.